

Inheritance

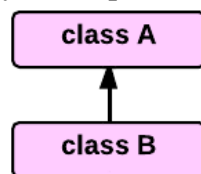
- ❖ Inheritance is the process of writing new class by inheriting commonly used state and behaviour of existing class.
- ❖ Existing Class is called as super class or Base class or Parent class.
- ❖ Newly defined class is called as Sub class or Derived class or Child class.
- ❖ Inheritance is an important property or Derived class or Child class.
- ❖ Main goal of important property or features of OOPS.
- ❖ Main goal of inheritance is code reusability i.e. Inheritance can be used to re-use the properties and operations of the existing in the newly defined class.

Type of Inheritance

1. Simple Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance
5. Hybrid Inheritance

1. Simple Inheritance

There will be only one super class and one sub class.

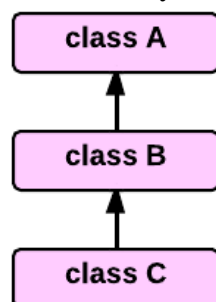


Example:

```
class A{...}  
class B extends A {...}
```

2. Multilevel Inheritance

- One super class can have only one direct sub class and many indirect sub classes.
- One sub class can have only one direct super class and many indirect super classes.

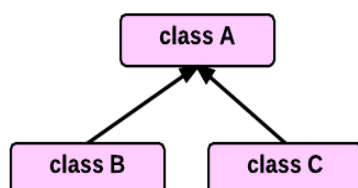


Example:

```
class A{...}  
  
class B extends A {...}  
class C extends B {...}
```

3. Hierarchical Inheritance

- One super class can have many direct sub classes.
- One sub class can have only one direct super class.

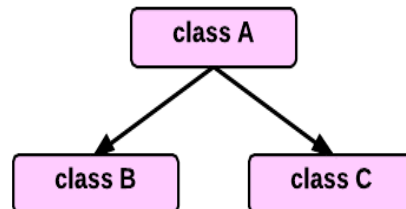


Example:

```
class A{...}  
  
class B extends A {...}  
class C extends A {...}
```

4. Multiple Inheritance

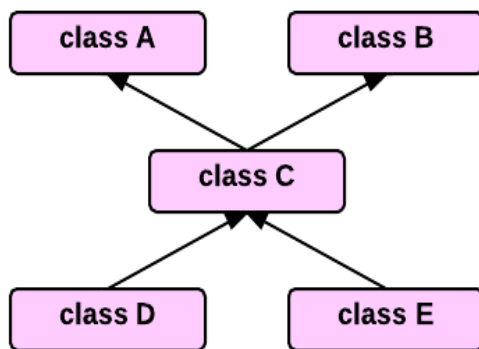
- One sub class can have many direct super classes.
- One super class can have only one direct sub class.
- Java does **not support** multiple inheritances with classes.
- Java supports multiple inheritances with interfaces.



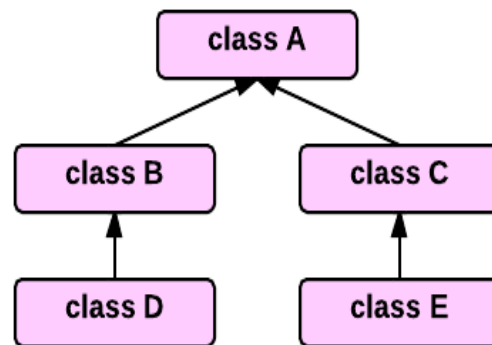
Example:
 class A {...}
 class B {...}
 class c extends A,B{...}

5. Hybrid Inheritance

- Hybrid Inheritance is the combination of two or more inheritance type.
- Java does not support Hybrid inheritance which is using multiple inheritances.



Combination of multiple inheritance,
 Hierarchical inheritance and Multilevel
 inheritance.
Invalid



Combination of Hierarchical inheritance
 and Multilevel inheritance.
Valid

Note:-

1. You can use extends keyword to inherit super class functionalities in sub class.
2. You can access only super class members by using super class object.
3. You can access super class members as well as sub class members by using sub class object.
4. All the super class members can be accessed from super class directly.
5. Object is the default super class for all the java classes.
6. When a class is not extending any class then Object class becomes direct super class.

```
class Hello{ } // Object is direct super class.
```
7. When a class is extending any super class then Object class becomes indirect super class.

```
class Hai{ }
class Hello extends Hai{ } // Object is indirect super class
```
8. Java does not support cyclic inheritance.
 Example: -1. class A extends A{ }
 2. class A extends B{ }
 class B extends A{ }

9. final class cannot be sub class.
10. Private members of super class will not be inherited to sub class.
11. When JVM loads the class then:
 - ⇒ It checks whether super class is loaded or not.
 - ⇒ If super class is not loaded then it loads super class then subclass.
 - ⇒ If super class is already loaded then it loads the subclass directly.
12. When you access static members of super class using subclass name then subclass will not be loaded.
13. When JVM create the object then:
 - ⇒ It allocates the memory of instance variable of super class.
 - ⇒ It allocates the memory of instance variable of subclass.
 - ⇒ It executes the instance blocks and constructors of super class.
 - ⇒ It executes the instance blocks and constructors of subclass.
14. When super class contains main () method then it will be inherited to sub class also i.e. you can execute java program either with super class name or sub class name.
15. Constructors of super class will not be inherited to subclass.
16. You can use same name for super class variables, sub class variable or local variables.
17. When you access any variable directly then
 - ⇒ Check whether that variable is declared in the local scope or not.
 - ⇒ If found in the local scope, then local variable will be used.
 - ⇒ If not found in the local scope then check that whether that variable is declared in the class scope or not.
 - ⇒ If found, that class level variable will be used.
 - ⇒ If not found in the scope then checks whether that variable is inherited from super classes or not.
 - ⇒ If found, that inherited variable will be used.

Programs:

OOPS248.java

```
class OOPS248{
    public static void main(String args[]){

        Hai hai=new Hai();
        System.out.println(hai. a);
        System.out.println(hai. b);
        Hello hello=new Hello();
        System.out.println(hello. a);
        System.out.println(hello. b);
    }
}
class Hai {
    int a=99;
}
class Hello {
    int b=88;
}
```

OOPS249.java

```
class OOPS249{
    public static void main(String args[]){

        Hai hai=new Hai();
        System.out.println(hai. a);
        System.out.println(hai. b);
        Hello hello=new Hello();
        System.out.println(hello. a);
        System.out.println(hello. b);
    }
}
class Hai {
    int a=99;
}
class Hello {
    int b=88;
}
```

OOPS250.java

```

class OOPS250{
    public static void main(String args[]){
        Hai hai=new Hai();
        System.out.println(hai.a);
        Hello hello=new Hello();
        System.out.println(hello.a);
        System.out.println(hello.b);
    }
}
class Hai {
    int a=99;
}
class Hello extends Hai{
    int b=88;
}

```

OOPS251.java

```

class OOPS251{
    public static void main(String args[]){
        Hello hello=new Hello();
        hello.m1();
        hello.m2();
    }
    class Hai {
        int a=99;
        void m1(){
            System.out.println("hai->m1:"+a);
            System.out.println("hai->m1:"+b);
        }
    }
    class Hello extends Hai{
        int b=88;
        void m2(){
            System.out.println("Hello->m2:"+a);
            System.out.println("Hello->m2:"+b);
        }
    }
}

```

OOPS252.java

```

class OOPS252{
    public static void main(String args[]){
        Hello hello=new Hello();
        hello.m1();
        hello.m2();
    }
}
class Hai {
    int a=99;
    void m1(){
        System.out.println("hai->m1:"+a);
    }
}
class Hello extends Hai{
    int b=88;
    void m2(){
        System.out.println("hai->m2:"+a);
        System.out.println("hai->m2:"+b);
    }
}

```

OOPS253.java

```

class OOPS253{
    public static void main(String args[]){
        C cobj=new C();
        cobj.m1();
        cobj.m2();
        cobj.m3();
    }
    class A{
        void m1(){
            System.out.println("A->m1 ()");
        }
    }
    class B extends A{
        void m2() {
            System.out.println("B->m2 ()");
        }
    }
    class C extends B{
        void m3() {
            System.out.println("C->m3 ()");
        }
    }
}

```

OOPS254.java

```

class OOPS254{
    public static void main(String args[]){
        B bobj=new B();
        bobj.m1();
        bobj.m2();
        C cobj=new C();
        cobj.m1();
        cobj.m3();
    }
    class A{

```

```

        void m1(){
            System.out.println("A->m1 ()");
        }
    }
    class B extends A{
        void m2() {
            System.out.println("B->m2 ()");
        }
    }
    class C extends A{
        void m3() {
            System.out.println("C->m3 ()");
        }
    }
}

```

OOPS255.java

```
class OOPS255{
    public static void main(String args[]){
        System.out.println("Hello guys");
    }
    class A{ }
    class B{ }
    class C extends A,B{ }
```

OOPS256.java

```
class OOPS256{
    public static void main(String args[]){
        System.out.println("Hello guys");
    }
    class Hai{ }
    class Hello extends Object, Hai{ }
```

OOPS257.java

```
class OOPS257{
    public static void main(String args[]){
        Hello hello=new Hello();
        Hai hai=new Hai();
        System.out.println(hai.toString());
        System.out.println(hello.toString());
    }
}
class Hai{ }
class Hello extends Hai{ }
```

OOPS258.java

```
class OOPS258{
    public static void main(String args[]){
        Hello hello=new Hello();
        hello.m1();
        hello.m2();
        System.out.println(hello.toString());
    }
    class Hai {
        void m1(){
            System.out.println("hai->m1 ()");
        }
    }
    class Hello extends Hai{
        void m2(){
            System.out.println("hai->m2 ()");
        }
    }
}
```

OOPS259.java

```
class OOPS259{
    public static void main(String args[]){
        System.out.println("Hello guys");
    }
}
class A extends A{ }
```

OOPS260.java

```
class OOPS260{
    public static void main(String args[]){
        System.out.println("Hello guys");
    }
    class A extends B{ }
    class B extends A{ }
```

OOPS261.java

```
class OOPS261{
    public static void main(String args[]){
        System.out.println("Hello guys");
    }
    final class A{}
    class B extends A{ }
```

OOPS262.java

```
class OOPS262{
    public static void main(String args[]){
        Hello hello=new Hello();
        hello.show();
    }
    class Hai{
        private int a=99; }
    class Hello extends Hai{
        void show(){
            System.out.println(a);
        }
    }
}
```

OOPS263.java

```
class OOPS263{
    public static void main(String args[]){
        Hello hello=new Hello();
        hello.show();
    }
    class Hai{
        int b=88;
```

```
protected int c=77;
public int d=66;
}
class Hello extends Hai{
    void show(){
        System.out.println(b);
        System.out.println(c);
        System.out.println(d);
    }
}
```

Inheritance and Blocks:

<u>OOPS264.java</u> <pre> class OOPS264{ public static void main(String args[]){ new Hello(); } } class Hai{ static { System.out.println("Hai->S.B"); } } class Hello extends Hai{ static { System.out.println("Hello->S.B"); } } </pre>	<u>OOPS265.java</u> <pre> class OOPS265{ public static void main(String args[]){ new Hello(); } } class Hai{ static int a=99; static { System.out.println("Hai->S.B : "+a); } } class Hello extends Hai{ static int b=88; static { System.out.println("Hello->S.B : "+a); System.out.println("Hello->S.B : "+b); } } </pre>
<u>OOPS266.java</u> <pre> class OOPS266{ public static void main(String args[]){ System.out.println(Hello.a); } } class Hai{ static { System.out.println("Hai -> S.B "); } } class Hello extends Hai{ static int a=99; static { System.out.println("Hello->S.B "); } } </pre>	<u>OOPS267.java</u> <pre> class OOPS267{ public static void main(String args[]){ System.out.println(Hai.a); } } class Hai{ static int a=99; static { System.out.println("Hai->S.B "); } } class Hello extends Hai{ static { System.out.println("Hello->S.B "); } } </pre>
<u>OOPS268.java</u> <pre> class OOPS268{ public static void main(String args[]){ System.out.println(Hello.a); } } </pre>	<pre> class Hai{ static int a=99; static { System.out.println("Hai->S.B "); } } class Hello extends Hai{ static { System.out.println("Hello->S.B "); } } </pre>

OOPS269.java

```

class OOPS269{
    public static void main(String args[]){
        Hello h=new Hello();
    }
    class Hai{
    {
        System.out.println("Hai -> I.B ");
    }
    }
    class Hello extends Hai{
    {
        System.out.println("Hello->I.B");
    }
    }
}

```

OOPS270.java

```

class OOPS270{
    public static void main(String args[]){
        new Hello();
    }
    class Hai{
        int a=99;
    {
        System.out.println("Hai->I.B : "+a);
    }
    }
    class Hello extends Hai{
        int b=88;
    {
        System.out.println("Hello->I.B : "+a);
        System.out.println("Hello->I.B : "+b);
    }
    }
}

```

OOPS271.java

```

class OOPS271{
    public static void main(String args[]){
        Hello h=new Hello();
    }
    }
    class Hai{
    {
        System.out.println("Hai->I.B ");
    }
    static{
        System.out.println("Hai->S.B ");
    }
    }
    class Hello extends Hai{
    {
        System.out.println("Hello->I.B ");
    }
    static {
        System.out.println("Hello->S.B");
    }
    }
}

```

OOPS272.java

```

class Hai{
    public static void main(String args[]){
        System.out.println("Hai->main ");
    }
    static{
        System.out.println("Hai->S.B ");
    }
    }
    class OOPS272 extends Hai {
        static {
            System.out.println("OOPS802->S.B");
        }
    }
}

```

OOPS273.java

```

class OOPS273{
    public static void main(String args[]){
        Hello.show();
    }
    }
    class Hai{
        static void show(){
            System.out.println("Hai->show ");
        }
    }
}

```

```

static {
    System.out.println("Hai->S.B");
}
}
class Hello extends Hai{
    static{
        System.out.println("Hello->S.B ");
    }
}
}

```

Inheritance and Constructors:

OOPS274.java

```
class OOPS274{  
    public static void main(String args[]){  
        new A();  
    }  
    class A{  
        A(){  
            System.out.println("A->DC ");  
        }  
    }  
    class B extends A{  
        B(){  
            System.out.println("B->DC ");  
        }  
    }  
}
```

OOPS275.java

```
class OOPS275{  
    public static void main(String args[]){  
        new B();  
    }  
    class A{  
        A(){  
            System.out.println("A->DC ");  
        }  
    }  
    class B extends A{  
        B(){  
            System.out.println("B->DC ");  
        }  
    }  
}
```

OOPS276.java

```
class OOPS276{  
    public static void main(String args[]){  
        new C();  
    }  
    class A{  
        A(){  
            System.out.println("A->DC ");  
        }  
    }  
}
```

```
class B extends A{  
    B(){  
        System.out.println("B->DC ");  
    }  
    class C extends B{  
        C(){  
            System.out.println("C->DC ");  
        }  
    }  
}
```

OOPS277.java

```
class OOPS277{  
    public static void main(String args[]){  
        new C();  
    }  
    class A{  
        A(){  
            System.out.println("A->DC ");  
        }  
        static{  
            System.out.println("A->S.B ");  
        }  
        {  
            System.out.println("A->I.B ");  
        }  
    }  
}
```

```
class B extends A{  
    B(){  
        System.out.println("B->DC ");  
    }  
    static{  
        System.out.println("B->S.B ");  
    }  
    {  
        System.out.println("B->I.B ");  
    }  
    class C extends B{  
        C(){  
            System.out.println("B->DC ");  
        }  
        static{  
            System.out.println("C->S.B ");  
        }  
        {  
            System.out.println("C->I.B ");  
        }  
    }  
}
```

Program output /error and details:-

Prog No.	Output	Error and Details
OOPS248	Compilation Error	[Error: b cannot be resolved or is not a field a cannot be resolved or is not a field] Here, b is member of Hello class and a is member of Hai class. so you can't call b with Hai and a with Hello class reference.
OOPS249	Compilation Error	[Error : b cannot be resolved or is not a field a cannot be resolved or is not a field] Here, Hello extends Hai so you can call member of Hai but you can't call member of Hello class with class reference.
OOPS250	99 99 88	Here, Hello extends Hai so you can call member of Hai but you can't call member of Hello class with class reference.
OOPS251	Compilation Error	[Error :b cannot be resolved to a variable] Here, b is member of subclass. So it will not call in super class.
OOPS252	99 99 88	You can access only super class members by using super class object.
OOPS253	A->m1 () B->m2 () C->m3 ()	You can access super class member and sub class member using sub class object.
OOPS254	A->m1 () B->m2 () A->m1 () C->m3 ()	You can access super class member and sub class member using sub class object. Here, you are not calling m2 method show it will not display.
OOPS255	Compilation Error	[Error: - {expected} Java does not support multiple inheritances with classes.
OOPS256	Compilation Error	[Error: - {expected} Java does not support multiple inheritances with classes.
OOPS257	Hai@1f33675 Hello@7c6768	Here, toString () method is inherited from object class. it will display address of the class.
OOPS258	hai->m1 () hai->m2 () Hello@9931f5	You can access super class member and sub class member using sub class object and toString() method is display address of class.
OOPS259	Compilation Error	[Error : cyclic inheritance involving A] Java does not support cyclic inheritance.
OOPS260	Compilation Error	[Error : same ""]
OOPS261	Compilation Error	[Error: cannot inherit from final A] final class cannot be sub class.
OOPS262	Compilation Error	[Error : a has private access in Hai] Private members of super class will not be inherited to sub class.
OOPS263	88 77 66	you can access super class members (default, protected and public) and subclass member using subclass.

Inheritance and Blocks

Prog No.	Output	Error and Details
OOPS264	Hai->S.B Hello->S.B	1 st super class is loaded then subclass then 1 st static members of super class then static members of sub class will executed when you are creating object for subclass.
OOPS265	Hai->S.B : 99 Hello->S.B : 99 Hello->S.B : 88	Same""
OOPS266	Hai -> S.B Hello->S.B 99	Same"". But main method member is executing last only.
OOPS267	Hai->S.B 99	When you access static members of super class using subclass name then subclass will not be loaded.
OOPS268	Hai->S.B 99	We are not using any member of sub class member so subclass will not load.
OOPS269	Hai -> I.B Hello->I.B	Refer point 13.
OOPS270	Hai->I.B : 99 Hello->I.B : 99 Hello->I.B : 88	Same""
OOPS271	Hai->S.B Hello->S.B Hai->I.B Hello->I.B	1 st super class is loaded then subclass then 1 st static block of super class then static block of sub class will executed then Instance Block of super class then instance block of subclass will execute.
OOPS272	Hai->S.B OOPS802 -> S.B Hai->main	When super class contains main () method then it will be inherited to sub class also
OOPS273	Hai->S.B Hai->show	1 st static block then Static member and you are not using any member of subclass so it not loads.

Inheritance and Constructors

Prog No.	Output	Error and Details
OOPS274	A->DC	Refer point 15
OOPS275	A->DC B->DC	Constructors of super class will be inherited to sub class when object created for subclass.
OOPS276	A->DC B->DC C->DC	same ""
OOPS277	A->S.B B->S.B C->S.B A->I.B A->DC B->I.B B->DC C->I.B B->DC	First Static block will execute then instance block and Default Constructors of super class and then sub class.